UNIT-3

GREEDY METHOD

Syllabus: The Greedy Method: The General Method, Knapsack Problem, Job Sequencing with Deadlines, Minimum-cost Spanning Trees, Prim's Algorithm, Kruskal's Algorithms, Optimal Merge Patterns, Single Source Shortest Paths.

1. General method:

- The greedy method is a simple strategy of progressively building up a solution, one element at a time, by choosing the best possible element at each stage.
- At each stage, a decision is made regarding whether or not a particular input is in an optimal solution.
- This is done by considering the inputs in an order determined by some selection procedure.

Greedy technique follows three principles to make a decision

i. **Feasible:** Here we check whether it satisfies all possible constraints or not, to obtain at least one solution to our problems.
ii. **Local Optimal Choice:** In this, the choice should be the optimum which is selected from the currently available
iii. **Irrevocable or Unalterable:** Once the decision is made, at any subsequence step that option is not altered.

Algorithm for Greedy approach:

```
Algorithm Greedy
// a[1.. n] contains the 'n' inputs
{
 solution :=ϕ ;   // initialize the solution to empty
for i :=1 to n do
          {
           x := select (a)
           if feasible (solution, x) then
           solution := Union (solution, x);  // Gets solution set
           }
 return solution;
}
```

In Greedy method following activities are performed:

i) First we select some solution from input domain.

ii) Then we check whether the solution is feasible or not.

iii) From the set of feasible solutions, and select particular solution that satisifies or nearly satisfies the objective of the function such a solution is called optimal solution.

Feasible solution, Optimal solution and Objective function

Feasible solution: For solving a particular problem, there exists 'n' inputs and we need to obtain a subset that satisfies some constraints. Then any subset that satisfies these constraints is called Feasible solution.

Optimal solution: From a set of feasible solutions , a particular solution that satisfies or nearly satisfies the objectives of the function, such a solution is called Optimal solution.

Objective function: A feasible solution that either minimizes or maximizes a given objective function is called Objective function.

Comparison between Divide and conquer approach and Greedy approach

Divide and conquer approach	Greedy approach
It is used to obtain a solution to the given problem.	It is used to obtain optimal solution.
In this technique, the problem is divided into smaller subproblems. These subproblems are solved independently. Finally all the solutions of subproblems are collected together, to get the final solution to the given problem.	In Greedy method, a set of feasible solutions is generated and optimum solution is picked up.
In this method, duplications in subsolutions are neglected. That means duplicate solutions may be obtained.	In greedy method, the optimum selection is takes place without revisiting previously generated solutions.
Divide and conquer is less efficient because of rework on solutions.	Greedy method is comparatively efficient but there is no as such guarantee of getting optimum solution.
Examples: Quick sort, Binary search.	Examples: Knapsack problem, Finding minimum spanning tree.

2. Knapsack Problem(Fractional Knapsack Problem):

A list of items are given, each item has its own profit and weight. Items can be placed in a knapsack whose maximum weight limit is W. The problem is to find the weight that is less than or equal to W, and profit is maximized.

The knapsack problem can be solved by following the steps below:

1.Compute the value of profits per weights i.e., p/w

2.By the maximization property of greedy method, arrange the values from higher p/w to the lower and their corresponding values.

3. Taking the objects into knapsack from starting by subtracting the weight of the object from the knapsack capacity. When knapsack capacity is 0 no object can be taken into knapsack.

4.When weight of object is less than capacity of knapsack then x_i=1.0 and when weight of object is greater than capacity of knapsack then $x_i=\left[\frac{capacity\ of\ knapsack}{weight\ of\ object}\right]$.

5.Finally calculate the value of $\sum x_i p_i$ and $\sum x_i w_i$

$$\sum x_i p_i = x_1 p_1 + x_2 p_2 + \ldots + x_N p_N$$

$$\sum x_i w_i = x_1 w_1 + x_2 w_2 + \ldots + x_N w_N$$

Eg:

Consider knapsack capacity w=15 and 7 objects with respective weights and profits:

O={O1,O2,O3,O4,O5,O6,O7}

w={2,3,5,7,1,4,1}

p={10,5,15,7,6,18,3}

Object	O1	O2	O3	O4	O5	O6	O7
Profit(p)	10	5	15	7	6	18	3
Weight(w)	2	3	5	7	1	4	1
p/w	5	1.67	3	1	6	4.5	3

Arranging the objects in the decreasing order of p/w value

Object	O5	O1	O6	O3	O7	O2	O4
Profit(p)	6	10	18	15	3	5	7
Weight(w)	1	2	4	5	1	3	7
p/w	6	5	4.5	3	3	1.67	1
x_i	1	1	1	1	1	2/3	0

$$\sum x_i p_i = 6 + 10 + 18 + 15 + 3 + \frac{2}{3}(5)$$

$$= 55.34$$

$$\sum x_i w_i = 1 + 2 + 4 + 5 + 1 + (3 * \frac{2}{3})$$

$$= 15$$

Algorithm for Knapsack:

Algorithm GreedyKnapsack (m, n)

```
// P[1 : n] and w[1 : n] contain the profits and weights respectively of
// Objects ordered so that p[i] / w[i]> p[i + 1] / w[i + 1].
// m is the knapsack size and x[1: n] is the solution vector.
 {
   for i := 1 to n do x[i] := 0.0 // initialize x
        U := m;
        for i := 1 to n do
         {
           if (w(i) > U) then break;
               x [i] := 1.0; U := U – w[i];
         }
         if (i < n) then x[i] := U / w[i];
  }
```

3. Job sequencing with Deadlines

The objective of job sequencing is to find a sequence of jobs, which is completed within their deadlines and gives the maximum profit.

Method for solving job sequencing problem:

1.The number of jobs can be done is based on the maximum value of deadlines in the given jobs.
2.Arrange the profits in the descending order and choose the jobs that can be done within their deadlines.
3.Finally calculate the total profit.

Eg :

Consider the number of jobs is 7, their profits and deadlines are as following;

j={j1,j2,j3,j4,j5,j6,j7}
p={5,20,25,30,15,12,35}
d={2,2,4,4,3,1,3}

arranging the jobs in the descending order of their profits:

Jobs	J7	J4	J3	J2	J5	J6	J1
Profits(p)	35	30	25	20	15	12	5
Deadlines(d)	3	4	4	2	3	1	2

The maximum value of deadlines is 4 so the number of jobs can be done is 4

Deadlines	**1**	**2**	**3**	**4**
Jobs	**J2**	**J3**	**J7**	**J4**
Profits	**20**	**25**	**35**	**30**

Profit=35+30+25+20

=110

The job sequence for the given problem is (J2,J3,J7,j4)

Algorithm for job sequencing:

```
Algorithm GreedyJobSeq (d,J,n)
// J is a set of jobs that can be completed by their deadlines.
{
  J := {1};
  for i := 2 to n do
   {
```

```
    if (all jobs in J U {i} can be completed by their dead lines) then
        J := J U {i};
    }
}
```

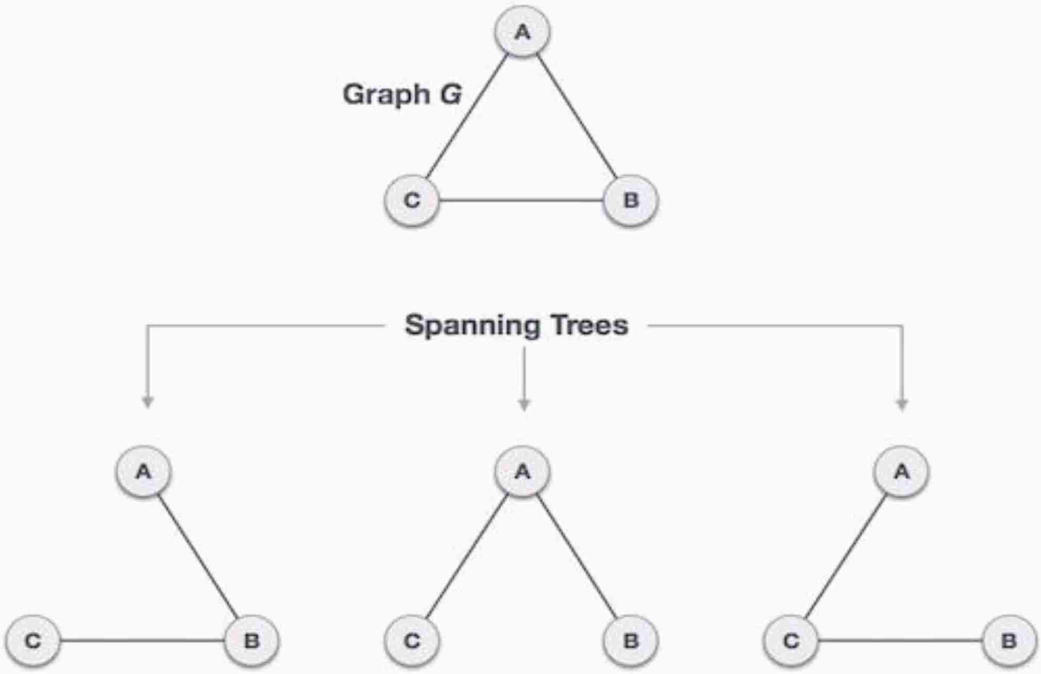
The **time-complexity** of this problem is $O(n^2)$

4. Minimum-cost Spanning Trees

Spanning Tree:

Spanning tree can be defined as a sub-graph of connected, undirected graph G that is a tree produced by removing the desired number of edges from a graph. In other words, Spanning tree is a non-cyclic sub-graph of a connected and undirected graph G that connects all the vertices together. A graph G can have multiple spanning trees and the vertices of a spanning tree and the graph given graph should be equal.

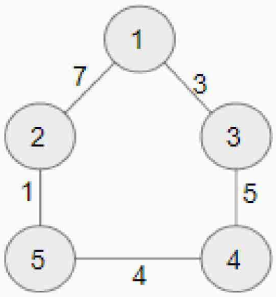
Eg:



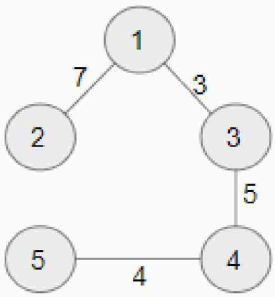
Minimum Spanning Tree

There can be weights assigned to every edge in a weighted graph. However, A minimum spanning tree is a spanning tree which has minimal total weight. In other words, minimum spanning tree is the one which contains the least weight among all other spanning tree of some particular graph.

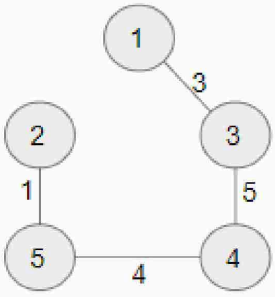
Eg: Consider a graph :



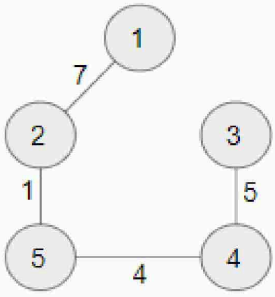
The possible spanning trees for the above graph are:



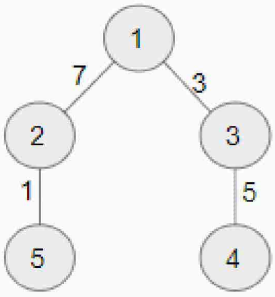
ST-1
w=19



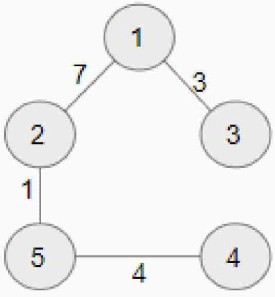
ST-2
w=13



ST- 3
w=17



ST-4
w=16



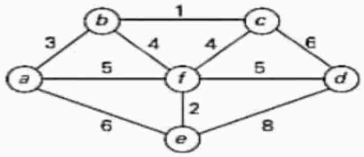
ST-5
w=15

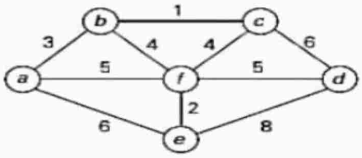
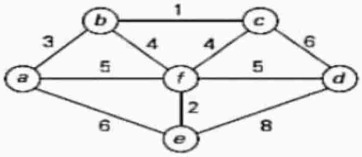
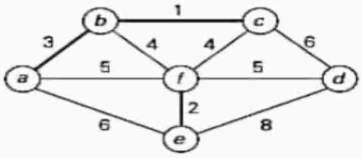
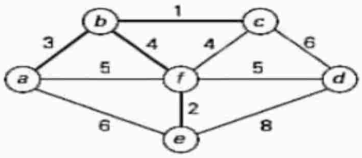
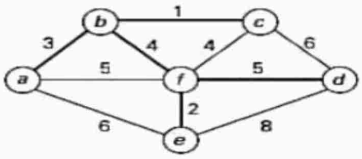
In the above spanning trees ST-2 is the MST as the weight is less when compared to the remaining spanning trees.

4a) Kruskal's method:

Kruskal's Algorithm is used to find the minimum spanning tree for a connected weighted graph. The main target of the algorithm is to find the subset of edges by using which, we can traverse every vertex of the graph. Kruskal's algorithm follows greedy approach which finds an optimum solution at every stage instead of focusing on a global optimum.

Eg : Compute the minimum spanning tree for the following graph using kruskal's method:



Tree edges	Sorted list of edges	Illustration
	bc ef ab bf cf af df ae cd de 1 2 3 4 4 5 5 6 6 8	
bc 1	bc **ef** ab bf cf af df ae cd de 1 2 3 4 4 5 5 6 6 8	
ef 2	bc ef **ab** bf cf af df ae cd de 1 2 3 4 4 5 5 6 6 8	
ab 3	bc ef ab **bf** cf af df ae cd de 1 2 3 4 4 5 5 6 6 8	
bf 4	bc ef ab bf cf af **df** ae cd de 1 2 3 4 4 5 5 6 6 8	
df 5		

Algorithm for Kruskal's method:

Algorithm Kruskal(G)
//Kruskal's algorithm for constructing a minimum spanning tree
//Input: A weighted connected graph G=(V, E)
//Output: E_T, the set of edges composing a minimum spanning tree of G sort E in nondecreasing order of the edge weights $w(e_{i_1}) <...< w(e_{i_{|E|}})$

$E_T \leftarrow \emptyset$; ecounter$\leftarrow$ 0 //initialize the set of tree edges and its size
k$\leftarrow$0 //initialize the number of processed edges
while ecounter < [V] - 1 do
k$\leftarrow$ k+1
if $E_T \cup e_{i_k}$ is acyclic

$E_T \leftarrow E_T \cup e_{i_k}$;
encounter ← encounter + 1

return E_T

The **Time-complexity** is O(E log |E|) where E is the number of Edges.

4b) Prims Algorithm:

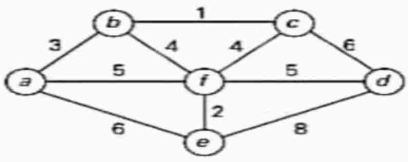
Prim's algorithm to find minimum cost spanning tree (as Kruskal's algorithm) uses the greedy approach. Prim's algorithm shares a similarity with the **shortest path first** algorithms.

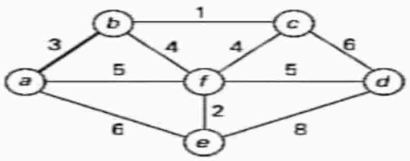
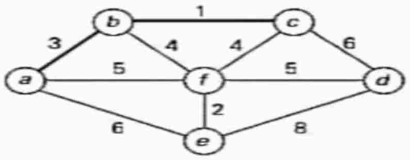
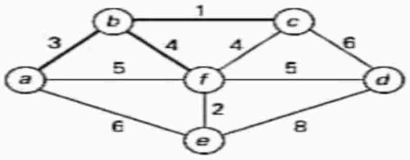
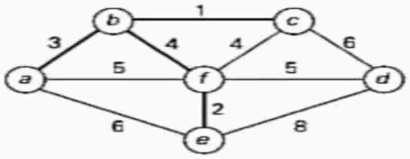
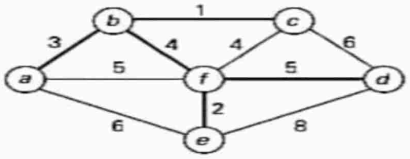
Prim's algorithm, in contrast with Kruskal's algorithm, treats the nodes as a single tree and keeps on adding new nodes to the spanning tree from the given graph.

Algorithm for prim's method:

Algorithm Prim(G)
//Prim's algorithm for constructing a minimum spanning tree
//Input: A weighted connected graph G = (V, E)
//Output: E_T, the set of edges composing a minimum spanning tree of G
$V_T \leftarrow$ {v0) //the set of tree vertices can be initialized with any vertex
$E_T \leftarrow \Phi$
for i=1 to |V| - 1 do
find a minimum-weight edge e* = (v*, u*) among all the edges (v, u) such
that v is inV_T , and u is in V - V_T
$V_T \leftarrow V_T \cup \{u^*\}$
$E_T \leftarrow E_T \cup \{e^*\}$
return E_T

Eg: Compute the minimum spanning tree for the following using prims algoritm:



Tree vertices	Remaining vertices	Illustration
a(−, −)	**b(a, 3)** c(−, ∞) d(−, ∞) e(a, 6) f(a, 5)	
b(a, 3)	**c(b, 1)** d(−, ∞) e(a, 6) f(b, 4)	
c(b, 1)	d(c, 6) e(a, 6) **f(b, 4)**	
f(b, 4)	d(f, 5) **e(f, 2)**	
e(f, 2)	**d(f, 5)**	
d(f, 5)		

The **Time-complexity** is O(E log |V|) where E is the number of Edges and V is the number of vertices .

Differences between prim's and kruskal's algorithm:

SNo.	Kruskal's algorithm	Prim's algorithm
1	Starts with all the vertices of the graph as a forest, and every addition of edge takes a forest one step further towards a complete tree.	Starts with any vertex of the graph arbitrarily no point of time, a forest is encountered in prim's algorithm.
2	The concept of kruskal's algorithm is based on the acyclic nature of the graph.	The concept of prim's algorithm is based on the connectedness.
3	More time is saved as the edges are sorted first.	Saves a lot of space instead, as no addition structures are used for storing the edges.

4	Adding of an edge is performed by selecting the sorted edge.	Always a new vertex is joined to an old vertex.
5	Next edge is always determined by the edge list.	Addition of nodes is based on the concept of shortest distance or weight.

5) Optimal Merge patterns:

In Optimal merge pattern, we merge a set of sorted files of different length into a single sorted file. We need to find an optimal solution, where the resultant file will be generated in minimum time.

Steps for optimal merge pattern:

1. Arrange the given files in ascending order.

2. Merge two files to a single file

3. If the remaining files are greater than that merged file then merge the next file with it.

4. If the remaining files are less than the merged file then merge the next two files and then combine the two merged files.

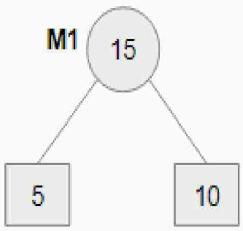
Eg: Consider files F1,F2,F3,F4,F5 with values 20,30,10,5,30 are given

Solution:

Arranging the files in ascending order:

F4	F3	F1	F2	F5
5	10	20	30	30

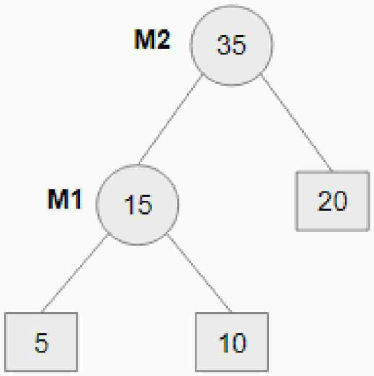
Step-1:



Remaining files:

F1(20),F2(30),F5(30)

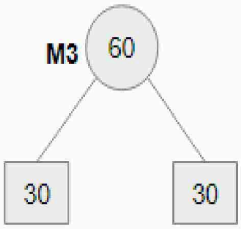
Step-2: M1<Remaining files so merging F1 and M1



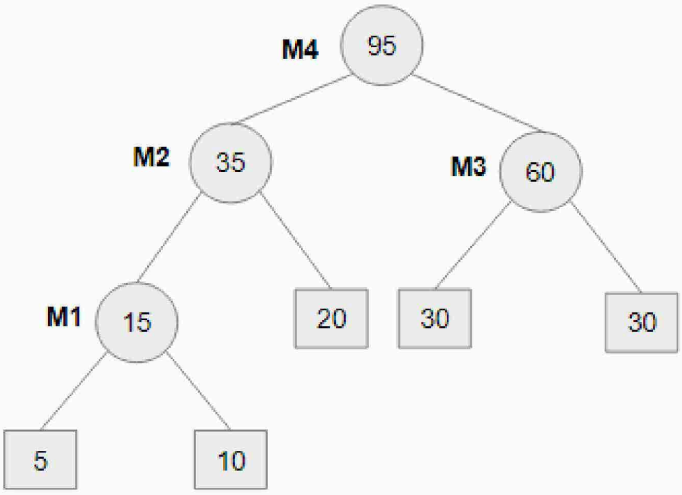
Remaining files:

F2(30),F5(30)

Step-3: M2>Remaining files so merge F2 and F5 (next two files)



Step-4: No files remaining so combine M3 and M2



Hence, the solution takes 15+35+60+95 = 205 number of comparisons.

Algorithm for Optimal tree by greedy approach:

```
Algorithm OptimalMerge(a[1…n])
//input : a set of m files or sorted lists.
//output : an optimal two way merge tree.
Begin
    Let Q be a primary queue.
        Q={initialize with all file lengths as weights}
        Repeat m-1 times //repeat till only one item is left.
          create a new node z
          x=extract_min(Q)
          y=extract_min(Q)
          left(z)=x
          right(z)=y
```

```
            weight(z)=weight(x)+weight(y)
            insert(Q,Z)
        end repeat
        return (extract_min)
end
```

The **time-complexity** for Optimal merge pattern problem is O(m log m), where m is the number of files.

6) Single source shortest paths method or Dijkstra's algorithm:

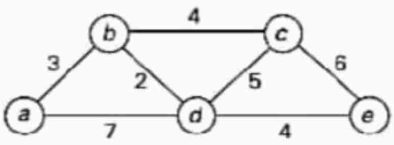
Dijkstra's algorithm is similar to prim's algorithm for MST. In prim's algorithm we generate MST, here we generate shortest path tree with a source as a root .

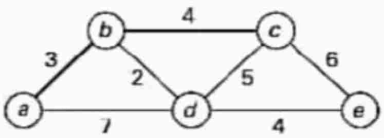
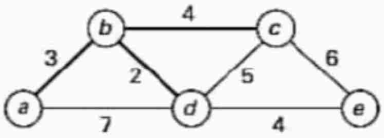
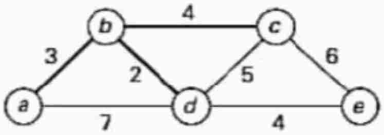
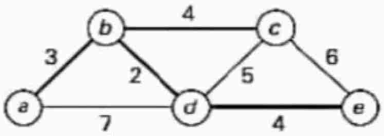
Algorithm for single source shortest path:

ALGORITHM Dijkstra(G, s)
//Dijkstra's algorithm for single-source shortest paths
//Input: A weighted connected graph G=(V, E) with nonnegative weights and its vertex s
//Output: The length d, of a shortest path from s to v and its penultimate vertex p, for every vertex v in V
Initialize(Q)
for every vertex v in V
$d_v \leftarrow \infty$; $p_v \leftarrow$ null
Insert(Q, v, d_v)
d← 0; Decrease(Q, s, d_s) //update priority of s with d_s
V← Φ
for i = 0 to |V| - 1 do
u* ← DeleteMin(Q) // delete the minimum priority element
$V_T \leftarrow$ V U {u*}
for every vertex u in V – $|V_T|$ that is adjacent to u* do
if d_{u^*} + w(u*, u) < d_u
$d_u \leftarrow d_{u^*}$ + w(u*, u); $p_u \leftarrow u^*$
Decrease(Q, u, d_u)

The time efficiency of Dijkstra's algorithm depends on the data structures used for implementing the priority queue and for representing an input graph itself. For graphs represented by their adjacency lists and the priority queue implemented as a min-heap, the **Time-complexity** is O(E log |V|) where E is the number of Edges and V is the number of vertices.

Eg: Find the shortest path tree for the given graph using dijkstra's algorithm



Tree vertices	Remaining vertices	Illustration
a(−, 0)	**b(a, 3)** c(−, ∞) d(a, 7) e(−, ∞)	
b(a, 3)	c(b, 3 + 4) **d(b, 3 + 2)** e(−, ∞)	
d(b, 5)	**c(b, 7)** e(d, 5 + 4)	
c(b, 7)	**e(d, 9)**	
e(d, 9)		

The shortest paths (identified by following nonnumeric labels backward from a destination vertex in the left column to the source) and their lengths (given by numeric labels of the tree vertices) are as follows:

from a to b:	$a - b$	of length 3
from a to d:	$a - b - d$	of length 5
from a to c:	$a - b - c$	of length 7
from a to e:	$a - b - d - e$	of length 9